

Linked Stack:

Code:

```
public class LinkedStack {
    private static class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }
    private Node top;
    private int size;

    public LinkedStack() {
        top = null;
        size = 0;
    }
    public void push(int data) {
        Node newNode = new Node(data);
        newNode.next = top;
        top = newNode;
        size++;
    }
    public int pop() {
        if (isEmpty()) {
            throw new RuntimeException("Stack underflow: stack is empty");
        }
        int data = top.data;
        top = top.next;
        size--;
        return data;
    }
    public int peek() {
        if (isEmpty()) {
            throw new RuntimeException("Stack is empty");
        }
        return top.data;
    }
    public int size() {
        return size;
    }
    public boolean isEmpty() {
        return size == 0;
    }
    public void printStack() {
```

```

Node current = top;
StringBuilder sb = new StringBuilder("[top] ");
while (current != null) {
    sb.append(current.data);
    if (current.next != null) sb.append(" -> ");
    current = current.next;
}
System.out.println(sb.toString());
}
public static void main(String[] args) {
    LinkedStack stack = new LinkedStack();
    stack.push(10);
    stack.push(20);
    stack.push(30);
    stack.printStack();
    System.out.println("Peek: " + stack.peek());
    System.out.println("Pop: " + stack.pop());
    stack.printStack();
    System.out.println("Size: " + stack.size());
    System.out.println("Empty: " + stack.isEmpty());
}
}

```

Output:

```

[top] 30 -> 20 -> 10
Peek: 30
Pop: 30
[top] 20 -> 10
Size: 2
Empty: false
PS C:\Users\Admin\Documents\DSA>

```

Linked Queue:

Code:

```

public class LinkedQueue {
    private static class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }
    private Node front;
    private Node rear;
    private int size;
}

```

```

public LinkedList() {
    front = null;
    rear = null;
    size = 0;
}
public void enqueue(int data) {
    Node newNode = new Node(data);
    if (rear == null) {
        front = rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
    size++;
}
public int dequeue() {
    if (isEmpty()) {
        throw new RuntimeException("Queue underflow: queue is empty");
    }
    int data = front.data;
    front = front.next;
    if (front == null) {
        rear = null; // queue became empty
    }
    size--;
    return data;
}
public int peek() {
    if (isEmpty()) {
        throw new RuntimeException("Queue is empty");
    }
    return front.data;
}
public int size() {
    return size;
}
public boolean isEmpty() {
    return size == 0;
}
public void printQueue() {
    Node current = front;
    StringBuilder sb = new StringBuilder("front -> ");
    while (current != null) {
        sb.append(current.data);
        if (current.next != null) sb.append(" -> ");
        current = current.next;
    }
}

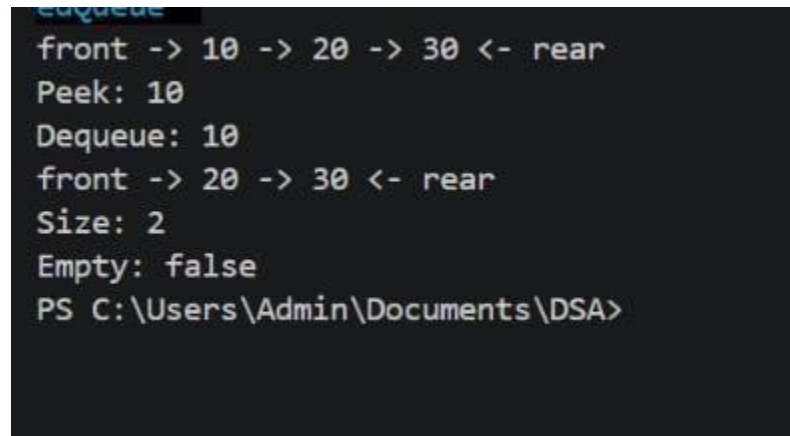
```

```

    }
    sb.append(" <- rear");
    System.out.println(sb.toString());
}
public static void main(String[] args) {
    LinkedList queue = new LinkedList();
    queue.enqueue(10);
    queue.enqueue(20);
    queue.enqueue(30);
    queue.printQueue();
    System.out.println("Peek: " + queue.peek());
    System.out.println("Dequeue: " + queue.dequeue());
    queue.printQueue();
    System.out.println("Size: " + queue.size());
    System.out.println("Empty: " + queue.isEmpty());
}
}

```

Output:



```

front -> 10 -> 20 -> 30 <- rear
Peek: 10
Dequeue: 10
front -> 20 -> 30 <- rear
Size: 2
Empty: false
PS C:\Users\Admin\Documents\DSA>

```

Circular Queue:

Code:

```

public class CircularQueue {
    private final int[] arr;
    private final int capacity;
    private int front;
    private int rear;
    private int size;
    public CircularQueue(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("Capacity must be positive");
        }
        this.capacity = capacity;
        this.arr = new int[capacity];
    }
}

```

```

        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }
    public void enqueue(int data) {
        if (isFull()) {
            throw new RuntimeException("Queue overflow: queue is full");
        }
        rear = (rear + 1) % capacity; // wrap around
        arr[rear] = data;
        size++;
    }
    public int dequeue() {
        if (isEmpty()) {
            throw new RuntimeException("Queue underflow: queue is empty");
        }
        int data = arr[front];
        front = (front + 1) % capacity; // wrap around
        size--;
        return data;
    }
    public int peek() {
        if (isEmpty()) {
            throw new RuntimeException("Queue is empty");
        }
        return arr[front];
    }
    public int size() {
        return size;
    }
    public boolean isEmpty() {
        return size == 0;
    }
    public boolean isFull() {
        return size == capacity;
    }
    public void printQueue() {
        StringBuilder sb = new StringBuilder("");
        for (int i = 0; i < size; i++) {
            sb.append(arr[(front + i) % capacity]);
            if (i < size - 1) sb.append(", ");
        }
        sb.append("]");
        System.out.println(sb.toString());
    }
    public static void main(String[] args) {

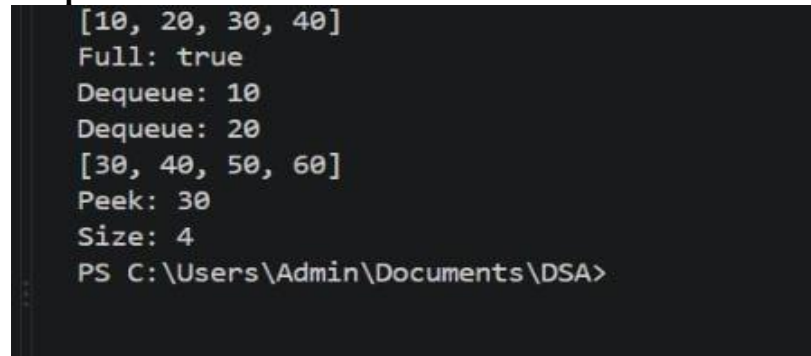
```

```

CircularQueue queue = new CircularQueue(4);
queue.enqueue(10);
queue.enqueue(20);
queue.enqueue(30);
queue.enqueue(40);
queue.printQueue();
System.out.println("Full: " + queue.isFull());
System.out.println("Dequeue: " + queue.dequeue());
System.out.println("Dequeue: " + queue.dequeue());
queue.enqueue(50);
queue.enqueue(60);
queue.printQueue();
System.out.println("Peek: " + queue.peek());
System.out.println("Size: " + queue.size());
}
}

```

Output:



```

[10, 20, 30, 40]
Full: true
Dequeue: 10
Dequeue: 20
[30, 40, 50, 60]
Peek: 30
Size: 4
PS C:\Users\Admin\Documents\DSA>

```

2D Array:

Code:

```

public class MyArray2D {
    private final int[] data;
    private final int rows;
    private final int cols;
    public MyArray2D(int rows, int cols) {
        if (rows <= 0 || cols <= 0) {
            throw new IllegalArgumentException("Rows and columns must be positive");
        }
        this.rows = rows;
        this.cols = cols;
        this.data = new int[rows * cols];
    }
    private int index(int row, int col) {
        if (row < 0 || row >= rows || col < 0 || col >= cols) {
            throw new IndexOutOfBoundsException("Invalid position (" + row + ", " + col + ")");
        }
        return row * cols + col;
    }
}

```

```

    }
    public void set(int row, int col, int value) {
        data[index(row, col)] = value;
    }
    public int get(int row, int col) {
        return data[index(row, col)];
    }
    public void fill(int value) {
        for (int i = 0; i < data.length; i++) {
            data[i] = value;
        }
    }
    public int[] getRow(int row) {
        int[] result = new int[cols];
        for (int c = 0; c < cols; c++) {
            result[c] = get(row, c);
        }
        return result;
    }
    public int[] getColumn(int col) {
        int[] result = new int[rows];
        for (int r = 0; r < rows; r++) {
            result[r] = get(r, col);
        }
        return result;
    }
    public int sum() {
        int total = 0;
        for (int value : data) total += value;
        return total;
    }
    public MyArray2D transpose() {
        MyArray2D result = new MyArray2D(cols, rows);
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                result.set(c, r, get(r, c));
            }
        }
        return result;
    }
    public int rows() {
        return rows;
    }
    public int cols() {
        return cols;
    }
}

```

```

public void print() {
    for (int r = 0; r < rows; r++) {
        StringBuilder sb = new StringBuilder("");
        for (int c = 0; c < cols; c++) {
            sb.append(get(r, c));
            if (c < cols - 1) sb.append(", ");
        }
        sb.append("]");
        System.out.println(sb.toString());
    }
}

public static void main(String[] args) {
    MyArray2D arr = new MyArray2D(2, 3);
    int value = 1;
    for (int r = 0; r < arr.rows(); r++) {
        for (int c = 0; c < arr.cols(); c++) {
            arr.set(r, c, value++);
        }
    }
    arr.print();
    System.out.println("Get (1, 2): " + arr.get(1, 2));
    System.out.println("Row 0: " + java.util.Arrays.toString(arr.getRow(0)));
    System.out.println("Col 1: " + java.util.Arrays.toString(arr.getColumn(1)));
    System.out.println("Sum: " + arr.sum());
    System.out.println("Transpose:");
    arr.transpose().print();
}
}

```

Code:

```

[1, 2, 3]
[4, 5, 6]
Get (1, 2): 6
Row 0: [1, 2, 3]
Col 1: [2, 5]
Sum: 21
Transpose:
[1, 4]
[2, 5]
[3, 6]
PS C:\Users\Admin\Documents\DSA>

```